

Как работают CRUD-операции

Информационная система городской телефонной сети

Подробный разбор: от клика в браузере до SQL и обратно

Содержание

1	Идея: один обобщённый CRUD на все сущности	2
2	Полный путь запроса (на примере «список абонентов»)	2
3	Каркас: crud_routes — 5 маршрутов одной функцией	2
4	READ — чтение	3
4.1	Список с пагинацией (GET /api/{resource})	3
4.2	Одна запись (GET /api/{resource}/{id})	4
5	CREATE — создание (POST /api/{resource})	4
6	UPDATE — изменение (PUT /api/{resource}/{id})	4
7	DELETE — удаление (DELETE /api/{resource}/{id})	5
8	Как сущности подключаются к CRUD	5
9	Права (RBAC)	5
10	Ошибки базы данных → понятные HTTP-ответы	6
11	Фронтенд: один компонент на все таблицы	6
12	Рецепт: добавить новую сущность в CRUD	7
13	Шпаргалка к защите (по CRUD)	7

1 Идея: один обобщённый CRUD на все сущности

CRUD = Create / Read / Update / Delete (создать / прочитать / изменить / удалить) — четыре базовые операции над данными. Плюс «список с постраничной выдачей» (**пагинация**).

В проекте 16 справочных сущностей (АТС и её 3 подтипа, абоненты, номера, адреса, города, звонки, тарифы, счета, платежи, пени, уведомления, очередь, таксофоны). Если писать CRUD для каждой отдельно — это 16 копий почти одинакового кода. Вместо этого написан **один обобщённый («generic») набор из 5 обработчиков**, который работает с **любой** сущностью. Это и есть выполнение требования «CRUD по всем сущностям + переиспользование кода».

Где смотреть. Серверная часть CRUD — всего два файла:

- backend/server/src/crud.rs — обобщённые обработчики (list/get_one/create/update/delete) и сборщик маршрутов crud_routes.
- backend/server/src/resources.rs — подключение каждой сущности к этому обобщённому коду.

Данные о таблицах (сущности SeaORM) — в backend/entity/src/*.rs. Клиентская часть — один компонент frontend/src/views/CrudView.vue + «описания» сущностей в frontend/src/config/resources.ts.

2 Полный путь запроса (на примере «список абонентов»)

Когда оператор открывает страницу «Абоненты», происходит следующее:

Шаг	Что происходит
1. Браузер	Vue отправляет HTTP-запрос GET /api/subscribers?page=1&page_size=20 (через axios, frontend/src/api/crud.ts).
2. Прокси	В разработке Vite, в Docker — nginx; перенаправляют /api/... на backend (порт 8080). Куки сессии едут вместе с запросом.
3. Роутер Ахум	main.rs монтирует всё под /api; resources.rs направляет /subscribers в обобщённый обработчик list::<subscriber::Entity>.
4. Проверка прав	Обработчик требует право subscriber:read. Нет права → ответ 403.
5. SeaORM	Entity::find().paginate(...) строит и выполняет SQL SELECT ... LIMIT 20 OFFSET 0 (+ запрос на общее число строк).
6. PostgreSQL	Возвращает строки; SeaORM раскладывает их в структуры Rust (Vec<Model>).
7. Ответ	Обработчик отдаёт JSON { items, total, page, page_size, total_pages }.
8. Браузер	Vue рисует таблицу и постраничную навигацию.

3 Каркас: crud_routes — 5 маршрутов одной функцией

Точка входа — обобщённая функция. Для сущности E она выдаёт Router с пятью маршрутами:

```
pub fn crud_routes<E>() -> Router<AppState>
where
    E: Resource + 'static,
    E::Model: Serialize + Send + Sync + IntoActiveModel<E::ActiveModel> + for<'de>
Deserialize<'de>,
    E::ActiveModel: ActiveModelTrait<Entity = E> + ActiveModelBehavior + Send + Sync +
Default,
    E::PrimaryKey: PrimaryKeyToColumn<Column = E::Column> + PrimaryKeyTrait<ValueType =
i64>,
{
```

```

Router::new()
    .route("/", get(list::<E>).post(create::<E>))
    .route("/{id}", get(get_one::<E>).put(update::<E>).delete(delete::<E>))
}

```

Как читать. `crud_routes<E>` — функция, **обобщённая по типу** `E` (entity-таблица). Блок `where { ... }` — это **ограничения на тип** `E`: «`E` должна быть таблицей SeaORM (Resource), её строка (Model) должна уметь превращаться в JSON (Serialize) и обратно (Deserialize), её первичный ключ — целое число (ValueType = i64)». Эти ограничения нужны, чтобы внутри можно было звать `find()`, `insert()`, `paginate()` и т.п. Маршруты:

Метод + путь	Обработчик	Операция
GET /	list	список (с пагинацией)
POST /	create	создать
GET /{id}	get_one	получить одну запись
PUT /{id}	update	изменить
DELETE /{id}	delete	удалить

Важно. `list::<E>` — это «подставить конкретную сущность в обобщённый обработчик». Например, `crud_routes::<subscriber::Entity>()` создаёт пять маршрутов именно для абонентов. Компилятор Rust генерирует отдельную версию кода под каждую сущность — это называется **мономорфизация**.

4 READ — чтение

4.1 Список с пагинацией (GET /api/{resource})

```

async fn list<E>(user: CurrentUser, State(st): State<AppState>, Query(p): Query<Pagination>)
-> AppResult<Json<Page<E::Model>>> { ...
    user.require(&format!("{}", E::NAME))?; // право <name>:read
    let page = p.page.max(1);
    let page_size = p.page_size.clamp(1, 200); // не больше 200 на страницу

    let mut select = E::find(); // SELECT * FROM таблица
    for pk in E::PrimaryKey::iter() {
        select = select.order_by_asc(pk.into_column()); // ORDER BY первичный ключ
    }
    let paginator = select.paginate(&st.db, page_size);
    let total = paginator.num_items().await?; // всего записей
    let total_pages = paginator.num_pages().await?;
    let items = paginator.fetch_page(page - 1).await?; // нужная страница (0-based)
    Ok(Json(Page { items, total, page, page_size, total_pages }))
}

```

Что важно понять:

- **Извлечение параметров.** `user: CurrentUser` — Axum сам достаёт пользователя из сессии (если не залогинен — 401). `Query(p)` — параметры из строки запроса (`page`, `page_size`). Если их нет — берутся значения по умолчанию (1 и 20).
- **Пагинация.** `paginate(db, page_size) + fetch_page(n)` SeaORM превращает в `LIMIT page_size OFFSET n*page_size`. Отдельно `num_items()` делает `SELECT COUNT(*)`, чтобы фронт знал, сколько всего страниц.

Сгенерированный SQL для абонентов, страница 1 по 20:

```

SELECT COUNT(*) FROM "subscriber"; -- num_items()
SELECT "subscriber"."id", "subscriber"."last_name", ... -- fetch_page(0)
FROM "subscriber" ORDER BY "subscriber"."id" ASC
LIMIT 20 OFFSET 0;

```

4.2 Одна запись (GET /api/{resource}/{id})

```
async fn get_one<E>(user, State(st), Path(id): Path<i64>) -> AppResult<Json<E::Model>> {
    user.require(&format!("{}",:read", E::NAME));
    let model = E::find_by_id(id).one(&st.db).await?.ok_or(AppError::NotFound)?;
    Ok(Json(model))
}
```

Path(id) — это часть URL (/api/subscribers/7 → id = 7). find_by_id(id).one(db) → SELECT ... WHERE id = 7 LIMIT 1. Если ничего не нашли (None) — ok_or(AppError::NotFound) превращает это в ответ 404.

5 CREATE — создание (POST /api/{resource})

```
async fn create<E>(user, State(st), Json(payload): Json<Value>) -> AppResult<Json<E::Model>>
{
    user.require(&format!("{}",:create", E::NAME));
    let mut am = E::ActiveModel::default(); // «пустая» редактируемая строка
    am.set_from_json(payload.clone()); // заполнить поля из присланного JSON
    if !E::PrimaryKey::auto_increment() { // для не-автоинкрементных ключей
        for pk in E::PrimaryKey::iter() { // (подтипы ATC: pbx_id) проставить ключ
            вручную
                let col = pk.into_column();
                if let Some(v) = payload.get(col.as_str()).and_then(|x| x.as_i64()) {
                    am.set(col, v.into());
                }
            }
        }
    }
    let model = am.insert(&st.db).await?; // INSERT ... RETURNING *
    Ok(Json(model))
}
```

Ключевое понятие — **ActiveModel** («активная модель»). Это «редактируемая» версия строки: у **каждого** поля есть состояние **Set** (значение задано) или **NotSet** (не трогаем — в БД сработает значение по умолчанию). Работает так:

- ActiveModel::default() — все поля NotSet.
- set_from_json(payload) — берёт присланный JSON и помечает Set только те поля, что в нём есть. Поэтому клиент может прислать **частичные** данные, а id, created_at, статусы и т.п. подставит сама БД (defaults).
- insert(db) — формирует INSERT INTO ... (только Set-поля) VALUES (...) RETURNING * и возвращает уже сохранённую строку (с присвоенным id).

Важно. Блок с auto_increment() нужен только для подтипов ATC (pbx_city/department/institution), где первичный ключ pbx_id задаёт клиент (это id существующей ATC), а не база. У обычных таблиц ключ id генерируется автоматически, и этот блок пропускается.

Пример: POST /api/cities с телом {"name":"Казань"} →

```
INSERT INTO "city" ("name") VALUES ('Казань') RETURNING "id", "name", "is_home",
"created_at";
```

is_home и created_at клиент не присылал — у них сработали значения по умолчанию из схемы.

6 UPDATE — изменение (PUT /api/{resource}/{id})

```
async fn update<E>(user, State(st), Path(id): Path<i64>, Json(payload): Json<Value>) -> ...
{
    user.require(&format!("{}",:update", E::NAME));
    let existing = E::find_by_id(id).one(&st.db).await?.ok_or(AppError::NotFound)?; // 1)
    найти
        let mut am = existing.into_active_model(); // 2) превратить в редактируемую
        am.set_from_json(payload); // 3) применить присланные поля
        let model = am.update(&st.db).await?; // 4) UPDATE ... WHERE id = ?
}
```

```
Ok(Json(model))
}
```

Логика: сначала **находим** запись (нет → 404), превращаем её в ActiveModel, накладываем присланные поля (set_from_json помечает Set только их — остальные остаются как были), и update(db) делает UPDATE таблица SET изменённые_поля WHERE id = ? RETURNING *. То есть можно прислать только то, что меняем.

7 DELETE — удаление (DELETE /api/{resource}/{id})

```
async fn delete<E>(user, State(st), Path(id): Path<i64>) -> AppResult<Json<Value>> {
    user.require(&format!("{}",:delete", E::NAME));
    let res = E::delete_by_id(id).exec(&st.db).await?; // DELETE FROM таблица WHERE id = ?
    if res.rows_affected == 0 {                       // ничего не удалили → не было
        такой записи
        return Err(AppError::NotFound);
    }
    Ok(Json(serde_json::json!({ "deleted": id })))
}
```

delete_by_id(id) → DELETE FROM таблица WHERE id = ?.rows_affected — сколько строк удалено; если 0, значит записи не было → 404.

Важно. Если на запись кто-то ссылается (внешний ключ с ON DELETE RESTRICT, например на абонента ссылается счёт), PostgreSQL не даст удалить и вернёт ошибку — мы превращаем её в понятный ответ 409 Conflict (см. раздел про ошибки).

8 Как сущности подключаются к CRUD

Чтобы обобщённый код узнал «имя» сущности (для прав), есть крошечный «интерфейс» Resource и макрос, который реализует его для каждой entity:

```
pub trait Resource: EntityTrait { const NAME: &'static str; }

macro_rules! resource {
    ($module:ident, $name:expr) => {
        impl Resource for entity::$module::Entity { const NAME: &'static str = $name; }
    };
}

resource!(subscriber, "subscriber");
resource!(pbx, "pbx");
resource!(city, "city");
// ...все 16
```

А дальше — список «путь → обобщённый CRUD для сущности»:

```
pub fn api_router() -> Router<AppState> {
    Router::new()
        .nest("/subscribers", crud_routes::<entity::subscriber::Entity>())
        .nest("/pbx",          crud_routes::<entity::pbx::Entity>())
        .nest("/cities",       crud_routes::<entity::city::Entity>())
        // ...и так все ресурсы
}
```

NAME («subscriber», «pbx», ...) используется для прав: subscriber:read, pbx:create и т.д.

9 Права (RBAC)

В каждом обработчике первая строка — проверка права вида сущность:действие:

```
user.require(&format!("{}",:read", E::NAME)); // в list / get_one
user.require(&format!("{}",:create", E::NAME)); // в create
```

```
user.require(&format!("{}",update", E::NAME)); // в update
user.require(&format!("{}",delete", E::NAME)); // в delete
```

CurrentUser подгружается из сессии вместе со списком прав (через роли). require пропускает дальше, если право есть (или пользователь – суперадмин), иначе возвращает 403 Forbidden. Роли и права настраиваются суперадмином в админке (таблицы role, permission).

10 Ошибки базы данных → понятные HTTP-ответы

Самое ценное: ограничения целостности (триггеры, CHECK, UNIQUE, внешние ключи) живут в БД. Если запрос их нарушает, PostgreSQL бросает ошибку, а файл error.rs превращает её в **осмысленный** HTTP-ответ, а не «500».

Нарушение	HTTP	Пример
Триггер (RAISE EXCEPTION)	400	Создать 2-го абонента на основном номере → «К номеру № 1 (тип линии «основной») можно подключить не более 1 абонент(ов)»
CHECK-ограничение	400	Простой абонент с указанной льготой → нарушение chk_privilege
UNIQUE	409	Номер телефона уже существует
Внешний ключ	409	Удаление записи, на которую ссылаются
Запись не найдена	404	GET/PUT/DELETE /{id} несуществующего id

Все сообщения – на русском: статические ошибки и коды PostgreSQL переводятся в error.rs (например 23505/UNIQUE → «Запись с такими данными уже существует»), а тексты триггеров заданы по-русски в самой БД. Поэтому форма на фронте показывает абоненту/оператору понятную причину отказа, а данные в БД остаются корректными при любом раскладе.

11 Фронтенд: один компонент на все таблицы

CRUD на клиенте тоже **обобщённый**. Есть один компонент CrudView.vue, который по «описанию» сущности рисует таблицу, пагинацию и форму создания/редактирования. Описания – в config/resources.ts:

```
{ key: 'subscribers', path: 'subscribers', title: 'Абоненты', perm: 'subscriber',
  columns: [ { prop: 'last_name', label: 'Фамилия' }, ... ],
  fields: [ { prop: 'last_name', label: 'Фамилия', type: 'text', required: true },
            { prop: 'category', label: 'Категория', type: 'select', options:
categoryOptions }, ... ] }
```

CrudView берёт текущий ресурс из URL, дёргает те же эндпоинты (GET/POST/PUT/DELETE /api/<path>), показывает кнопки «Добавить/Изменить/Удалить» только если у пользователя есть соответствующее право. Поле idField позволяет работать с подтипами АТС, у которых ключ называется pbx_id, а не id.

Поля-ссылки вместо ручного ввода id. Где раньше в форме вводили число (pbx_id, address_id, subscriber_id ...), теперь тип поля reference: CrudView подгружает связанный справочник и показывает **выпадающий список** с поиском и человекочитаемой подписью («12 · Иванов Иван»), а в форму уходит выбранный id. Те же подписи используются и в **таблице**: вместо «голых» id выводятся названия, enum-значения – русскими (active → «Активен»), а булевы – «Да/Нет». Так оператор не угадывает числа, а выбирает из списка.

Где смотреть.

- Таблица + форма: frontend/src/views/CrudView.vue
- Описания всех ресурсов: frontend/src/config/resources.ts
- Вызовы API: frontend/src/api/crud.ts

12 Рецепт: добавить новую сущность в CRUD

1. Создать таблицу в новой SQL-миграции (`backend/migrations/`).
2. Сгенерировать entity: `sea-orm-cli generate entity ...` → появится `entity/src/новая.rs`.
3. В `resources.rs`: дописать `resource!(новая, "новая")` и `.nest("/новая", crud_routes::<entity::новая::Entity>())`.
4. Добавить права `новая:read/create/update/delete` (миграция со вставкой в `permission` + выдача ролям).
5. На фронте: добавить описание в `config/resources.ts` — таблица, форма и меню появятся сами.

Готово — полноценный CRUD на бэке и на фронте без копирования логики.

13 Шпаргалка к защите (по CRUD)

Вопрос	Короткий ответ + где код
Как реализован CRUD по всем сущностям?	Один обобщённый <code>crud_routes::<E>()</code> на 16 ресурсов. <code>crud.rs</code> + <code>resources.rs</code> .
Где переиспользование кода?	5 generic-обработчиков работают с любой сущностью; подключение — списком в <code>resources.rs</code> .
Что такое ActiveModel?	Редактируемая строка: у каждого поля <code>Set/NotSet</code> ; <code>set_from_json</code> помечает только присланные поля → частичный ввод и работа default-ов БД.
Как сделана пагинация?	<code>paginate().fetch_page()</code> → <code>LIMIT/OFFSET</code> , плюс <code>num_items()</code> → <code>COUNT(*)</code> . Параметры <code>page</code> , <code>page_size</code> из строки запроса.
Как защищён доступ?	Каждый обработчик: <code>user.require("сущность:действие")</code> → иначе 403. Права из БД, настраивает суперадмин.
Что будет при нарушении правил БД?	Триггер/ЧЕК → 400, <code>UNIQUE/FK</code> → 409, нет записи → 404. Маппинг в <code>error.rs</code> .
Где здесь ORM (SeaORM)?	Именно в CRUD: <code>find</code> , <code>find_by_id</code> , <code>paginate</code> , <code>insert</code> , <code>update</code> , <code>delete_by_id</code> . Аналитика — на <code>sqlx</code> .

См. также общий разбор — `docs/guide.pdf`, схему БД — `docs/schema.dbml`, описание API — `docs/api.md`.